

```
suspend fun main() {
```

KOTLIN

COROUTINES

O Guia Prático para Desenvolvedores Android

Coroutines · suspend functions · Dispatchers

INICIANTE

INTERMEDIÁRIO

KOTLIN



Sumário

- | | |
|----|--|
| 01 | O Problema: Por que Coroutines? |
| 02 | Conceitos Fundamentais |
| 03 | suspend fun — A Magia Acontece Aqui |
| 04 | CoroutineScope e Builders |
| 05 | Dispatchers — Onde o Código Roda |
| 06 | Job, Cancelamento e Ciclo de Vida |
| 07 | async / await — Paralelismo |
| 08 | Coroutines no Android: ViewModel & Lifecycle |
| 09 | Tratamento de Erros |
| 10 | Cheat Sheet Rápido |

01 — O Problema: Por que Coroutines?

Apps Android rodam na **Main Thread** (UI Thread). Qualquer operação lenta nessa thread congela a tela e pode gerar o temido **ANR** (**A**pplicati**o**n **N**ot **R**espond**i**ng).

O problema clássico:

■ Bloqueante

```
// ■ ERRADO – Travando a Main Thread  
  
fun carregarDados() {  
    val dados = networkCall() // bloqueia por segundos!  
    textView.text = dados  
}
```

Antes das coroutines, a solução eram callbacks, AsyncTask (deprecated) ou RxJava. Todos têm sua complexidade. Coroutines chegaram para simplificar tudo isso.

Comparativo de abordagens:

Abordagem	Legibilidade	Cancelamento	Complexidade
Callbacks	Baixa	Manual	Alta
AsyncTask	Média	Difícil	Média
RxJava	Média/Alta	Boa	Alta
Coroutines ✓	Alta	Automático	Baixa

■ Coroutines são leves! Você pode criar milhares delas sem preocupação com memória, diferente de Threads.

02 — Conceitos Fundamentais

Antes de codar, entenda os três pilares:

suspend fun

Marca uma função como suspensível. Pode pausar sem bloquear a thread.

CoroutineScope

Define o ciclo de vida de uma coroutine. Quando o scope é cancelado, todas as coroutines filhas são canceladas.

Dispatcher

Define em qual thread/pool a coroutine vai executar (Main, IO, Default).

Como se relacionam:

Kotlin

```
// Anatomia de uma Coroutine
viewModelScope // 1. Scope – define ciclo de vida
.launch( // 2. Builder – inicia a coroutine
Dispatchers.IO // 3. Dispatcher – onde vai rodar
) {
val result = fetchData() // 4. suspend fun chamada aqui
_uiState.value = result
}
```

03 — suspend fun: A Magia Acontece Aqui

Uma **suspend function** pode ser *pausada* no meio da execução e *retomada* depois, sem bloquear a thread em que está rodando.

Kotlin

```
// Declarar uma suspend function

suspend fun buscarUsuario(id: String): Usuario {
    delay(1000) // pausa por 1s sem bloquear a thread
    return api.getUsuario(id)
}

// Chamar uma suspend function
// Só pode ser chamada de dentro de outra suspend fun
// ou de um coroutine builder (launch, async, runBlocking)

suspend fun carregarPerfil() {
    val usuario = buscarUsuario("42") // OK!
    exibirPerfil(usuario)
}
```

■ Regra de ouro: suspend functions só podem ser chamadas de outras suspend functions ou de um coroutine builder. O compilador vai te avisar se errar.

delay() vs Thread.sleep()

	delay()	Thread.sleep()
Bloqueia a thread?	■ Não	■ Sim
Pode ser cancelado?	■ Sim	■ Não
Onde usar?	Só em suspend/coroutine	Qualquer lugar

Kotlin

```
// delay() – suspende sem bloquear

suspend fun exemploDelay() {
    println("Antes")
    delay(2000) // suspende por 2 segundos
    println("Depois – a thread ficou livre!")
}
```

04 — CoroutineScope e Builders

O **CoroutineScope** é o contexto que gerencia coroutines. Os **builders** são as funções que criam e iniciam coroutines dentro de um scope.

launch { }

Inicia uma coroutine e retorna um Job. Não retorna resultado. Use para fire-and-forget.

async { }

Inicia uma coroutine e retorna um Deferred. Use quando precisar de resultado (com `.await()`).

runBlocking { }

Bloqueia a thread atual até a coroutine terminar. Use apenas em testes ou `main()` — nunca em produção Android.

coroutineScope { }

Cria um scope filho. Suspende até todos os filhos terminarem. Propaga exceções.

Exemplo completo com launch:

Kotlin · Activity

```
class MinhaActivity : AppCompatActivity() {  
    // lifecycleScope: cancelado quando a Activity é destruída  
    private fun carregarDados() {  
        lifecycleScope.launch {  
            // estamos na Main Thread aqui  
            showLoading(true)  
  
            val resultado = withContext(Dispatchers.IO) {  
                // mudou para IO Thread  
                repositorio.buscarDados()  
            }  
  
            // voltou para Main Thread automaticamente  
            showLoading(false)  
            exibir(resultado)  
        }  
    }  
}
```

05 — Dispatchers: Onde o Código Roda

O **Dispatcher** determina em qual thread (ou pool de threads) a coroutine vai executar. Escolher o Dispatcher certo é essencial para performance.

Dispatcher	Thread	Use para
Dispatchers.Main	UI Thread	Atualizar UI, observar LiveData, chamadas leves
Dispatchers.IO	Pool de IO (64 threads)	Rede, banco de dados, leitura de arquivos
Dispatchers.Default	Pool de CPU (núcleos)	Cálculos pesados, ordenação, JSON parsing
Dispatchers.Unconfined	Não confinado	Testes específicos — evite em produção

withContext() — Trocando de Dispatcher

Use **withContext()** para trocar de Dispatcher dentro de uma suspend function. É a forma idiomática e segura de fazer isso no Kotlin.

Kotlin · withContext

```
suspend fun salvarEExibir(item: Item) {  
    // Começa na thread que chamou (geralmente Main)  
  
    withContext(Dispatchers.IO) {  
        database.inserir(item) // roda na IO thread  
    }  
    // Voltou para a thread original (Main)  
  
    withContext(Dispatchers.Default) {  
        val processado = processarItem(item) // CPU intensivo  
    }  
    // Voltou para Main novamente  
  
    atualizarUI() // seguro aqui na Main  
}
```

■ Não precisa trocar para Main ao final — suspend functions respeitam o Dispatcher do scope que as chamou e voltam automaticamente.

06 — Job, Cancelamento e Ciclo de Vida

Toda coroutine retorna um **Job** — um handle para controlar sua execução. Você pode verificar o status, cancelar ou aguardar a conclusão.

Kotlin · Job

```
val job = lifecycleScope.launch {
    repeat(100) { i ->
        println("Iteração $i")
        delay(500)
    }
}

// Controlar o Job
job.cancel() // cancela a coroutine
job.join() // aguarda terminar (suspend)
job.isActive // true se ainda rodando
job.isCancelled // true se foi cancelada
job.isCompleted // true se terminou (normal ou cancelamento)
```

Cooperative Cancellation

O cancelamento em coroutines é **cooperativo**: a coroutine precisa verificar periodicamente se foi cancelada. Funções suspend já fazem isso automaticamente. Em loops longos, use **isActive**:

Kotlin · Cancelamento Cooperativo

```
val job = launch(Dispatchers.Default) {
    var count = 0

    // isActive verifica se a coroutine ainda está ativa
    while (isActive) {
        count++

        // sem delay() aqui, precisamos checar manualmente
        if (count > 1_000_000) break
    }

    delay(100)

    job.cancel() // a coroutine vai parar no próximo check de isActive
```

SupervisorJob — Isolando Falhas

Kotlin · SupervisorJob

```
// Com Job normal: se um filho falha, todos são cancelados
```



```
val scope = CoroutineScope(Job() + Dispatchers.Main)

// Com SupervisorJob: falha de um filho não afeta os outros
val supervisorScope = CoroutineScope(SupervisorJob() + Dispatchers.Main)

supervisorScope.launch { falhaAqui() } // falha
supervisorScope.launch { continuaRodando() } // continua OK
```

07 — async / await: Paralelismo Real

async inicia uma coroutine e retorna um **Deferred<T>**. Use **.await()** para aguardar o resultado. Isso permite executar operações *em paralelo*.

Sequencial vs Paralelo:

Kotlin · async/await

```
// ■ Sequencial – total: 2000ms

suspend fun carregarSequencial() {
    val usuario = buscarUsuario() // 1000ms
    val pedidos = buscarPedidos() // 1000ms
    combinar(usuario, pedidos)
}

// ■ Paralelo com async – total: ~1000ms

suspend fun carregarParalelo() = coroutineScope {
    val usuarioDeferred = async { buscarUsuario() } // inicia
    val pedidosDeferred = async { buscarPedidos() } // inicia

    // .await() aguarda o resultado de cada um
    val usuario = usuarioDeferred.await()
    val pedidos = pedidosDeferred.await()

    combinar(usuario, pedidos) // ambos já prontos!
}
```

■ Sempre use **async** dentro de **coroutineScope { }** para garantir que, se um **async** falhar, os outros sejam cancelados automaticamente.

Aguardando múltiplos resultados:

Kotlin · awaitAll

```
suspend fun carregarDashboard() = coroutineScope {
    val d1 = async { api.getMetrica1() }
    val d2 = async { api.getMetrica2() }
    val d3 = async { api.getMetrica3() }

    // awaitAll – aguarda todos e retorna lista
    val resultados = awaitAll(d1, d2, d3)
    montarDashboard(resultados)
}
```

08 — Coroutines no Android

O Android KTX fornece scopes prontos e integrados ao ciclo de vida. Nunca use **GlobalScope** em produção — ele não é cancelado com a Activity/Fragment.

viewModelScope	Cancelado quando o ViewModel é destruído. Ideal para operações de dados no ViewModel.
lifecycleScope	Cancelado quando a Activity/Fragment é destruída. Ideal para operações de UI.
repeatOnLifecycle	Coleta flows apenas quando o ciclo de vida está em um estado específico (STARTED, RESUMED).

Padrão recomendado com ViewModel:

Kotlin · ViewModel + Fragment

```
class ProdutosViewModel(private val repo: ProdutosRepo) : ViewModel() {  
    private val _produtos = MutableStateFlow<List<Produto>>(emptyList())  
    val produtos: StateFlow<List<Produto>> = _produtos  
  
    fun carregarProdutos() {  
        viewModelScope.launch { // cancelado com o ViewModel  
            try {  
                val lista = repo.buscarProdutos() // suspend fun  
                _produtos.value = lista  
            } catch (e: Exception) {  
                _produtos.value = emptyList()  
            }  
        }  
    }  
  
    // Na Activity/Fragment:  
    lifecycleScope.launch {  
        repeatOnLifecycle(Lifecycle.State.STARTED) {  
            viewModel.produtos.collect { lista ->  
                adapter.submitList(lista)  
            }  
        }  
    }  
}
```

09 — Tratamento de Erros

Coroutines propagam exceções de maneiras diferentes dependendo do builder usado. Entender isso evita bugs silenciosos.

Kotlin · try/catch

```
// try/catch – forma mais simples com launch
viewModelScope.launch {
    try {
        val dados = api.fetchDados()
        _state.value = Success(dados)
    } catch (e: IOException) {
        _state.value = Error("Sem conexão")
    } catch (e: HttpException) {
        _state.value = Error("Erro do servidor: ${e.code()}")
    }
}
```

CoroutineExceptionHandler — Capturando globalmente:

Kotlin · ExceptionHandler

```
val handler = CoroutineExceptionHandler { _, exception ->
    Log.e("Coroutine", "Erro não tratado: $exception")
    // Notificar usuário, reportar para crashlytics, etc
}

viewModelScope.launch(handler) {
    // se lançar exceção, o handler captura
    operacaoArriscada()
}
```

async e exceções:

Kotlin · async + exceções

```
// Com async, a exceção só é lançada no .await()
val deferred = async { operacaoArriscada() }

try {
    val resultado = deferred.await() // exceção lançada aqui
} catch (e: Exception) {
    // tratado aqui
}
```

■ ■ `CancellationException` é especial — não é considerado um erro. Se capturar em um `catch`, relance-a: `throw e`

10 — Cheat Sheet Rápido

O que fazer	Como fazer
Executar no background	<code>withContext(Dispatchers.IO) { }</code>
Iniciar e esquecer	<code>scope.launch { }</code>
Obter resultado assíncrono	<code>val r = async { }.await()</code>
Executar em paralelo	<code>async { } + async { } + awaitAll()</code>
Pausar sem bloquear	<code>delay(ms)</code>
Trocar para Main Thread	<code>withContext(Dispatchers.Main) { }</code>
Cancelar coroutine	<code>job.cancel()</code>
Verificar se foi cancelada	<code>isActive</code> ou <code>ensureActive()</code>
Scope no ViewModel	<code>viewModelScope.launch { }</code>
Scope na Activity	<code>lifecycleScope.launch { }</code>
Coletar Flow com lifecycle	<code>repeatOnLifecycle(State.STARTED) { collect{} }</code>
Tratar erros globalmente	<code>CoroutineExceptionHandler { _, e -> }</code>
Isolar falhas entre filhos	<code>SupervisorJob()</code>

Regras de Ouro

Nunca bloqueie a Main Thread	Use <code>withContext(Dispatchers.IO)</code> para operações de I/O
Nunca use <code>GlobalScope</code> em produção	Prefira <code>viewModelScope</code> ou <code>lifecycleScope</code>
Sempre trate <code>CancellationException</code>	Relance se capturar em um catch genérico
Use <code>coroutineScope{ }</code> dentro de <code>suspend fun</code>	Garante cancelamento estruturado
Prefira <code>withContext</code> a <code>launch</code> para resultados	<code>launch</code> é fire-and-forget

Próximos Passos

• Kotlin Flows — streams de dados assíncronos • Channel — comunicação entre coroutines • StateFlow e SharedFlow no ViewModel • Testes com `runTest` e `TestDispatcher`